

A Practical Guide to Evaluating AI Products

Expert-anchored failure analysis, automated evaluators, and an engineering roadmap, demonstrated on a clinical AI assistant

Dr. George T. Varghese, MD

June 2026

Abstract

Evaluations of large language model applications often produce many disconnected metrics that feel rigorous but rarely tell an engineering team what to fix. This paper presents a structured, expert-anchored alternative: a single domain expert defines what a correct answer requires, each output is scored Pass or Fail against a locked golden answer, failures are open-coded into an axial taxonomy of recurring mechanisms, and that taxonomy becomes a prioritized, verifiable engineering roadmap. The method is demonstrated on KallosumAI, a clinical AI assistant the author built for medical-affairs use, evaluated on a single drug (epoetin alfa) over a curated six-source corpus combined with live web search. Across a first iteration of 35 questions spanning five quality dimensions, failures were not diffuse but concentrated in the retrieval layer: source routing, retrieval, and answer specificity accounted for roughly half of all issues, while clinical safety, the strongest dimension, still carried rare but high-consequence misses. To show the method scaling beyond manual scoring, two automated evaluators were built and checked against the human labels: a deterministic code-based check found that 17 of 18 web-search answers cited a non-authoritative source, and an LLM-as-Judge for grounding, validated with a confusion matrix, proved systematically over-strict (perfect sensitivity, poor specificity), a concrete illustration of why a single agreement score misleads. The result is a reusable procedure that turns expert judgment into an actionable, re-runnable diagnosis rather than a single accuracy score.

1. Introduction and Hypothesis

We hypothesized that the failures of a clinical assistant combining retrieval-augmented generation with live web search cluster into a small number of recurring, addressable mechanisms. We further hypothesized that a structured, multi-dimensional evaluation, using expert-defined golden answers and axial-coded failure analysis, can surface those mechanisms and turn them into a prioritized engineering roadmap, more usefully than a single overall accuracy score. We demonstrate the method on KallosumAI, a clinical AI assistant the author built for medical-affairs use, evaluated on questions about a single drug, epoetin alfa; it is described in full in Section 4.

2. Related Work

Evaluation of LLM applications spans several established approaches. Standardized benchmarks (for example MMLU, or MedQA in the clinical domain) measure broad capability but say little about how a specific deployed product fails. General-purpose frameworks such as RAGAS and DeepEval offer reusable metrics for retrieval-augmented systems, and the LLM-as-Judge paradigm has become a common way to scale evaluation beyond manual review. The methodology used here follows the error-analysis-driven approach articulated by Hamel Husain

(Your AI Product Needs Evals, 2024): expert review of real traces, journaling failures as open codes, grouping them into axial categories, preferring code-based assertions where a failure can be checked deterministically, and reserving an LLM-as-Judge for subjective modes. For judge validation specifically, this approach draws on Shankar et al. (Who Validates the Validators?, UIST 2024), which argues that LLM-generated evaluators inherit the weaknesses of the models they assess and must therefore be validated against human grades before being trusted, using measures of coverage (true-negative rate) and false-failure rate. That work also identifies criteria drift, the finding that evaluation criteria cannot be fully specified before grading real outputs because grading itself refines the criteria. The present report applies this methodology end to end to a real clinical assistant: expert human scoring first, a domain-specific failure taxonomy, deterministic code-based checks where possible, and an LLM-as-Judge validated against human labels, carried through to a prioritized engineering roadmap. The contribution is not a new metric but a worked, honest application of these principles in a clinical setting.

3. The Evaluation Method

The method is organized as six steps, shown together in Figure 1 and described below, each a direct response to the failure modes of conventional AI evaluation. It is anchored in expert judgment, uses binary verdicts rather than rating scales, treats error analysis as the source of its metrics rather than an afterthought, and ends not in a dashboard but in a prioritized set of actions for the engineering team.

Step 1: Identify the principal domain expert

Evaluation begins by naming a single domain expert who defines what a correct output requires for each output type, including factual accuracy, completeness, structure, and tone. A single trusted owner, rather than a committee, keeps judgments consistent and the process fast enough to sustain. For a clinical assistant, that expert is a clinician.

Step 2: Collect traces and build the evaluation dataset

The expert assembles a set of representative traces of the application in use. A trace is the full record of one interaction, including the input, the context the system retrieved, any tool or search calls, intermediate steps, and the final output. For each trace, three things are captured alongside the output: the sources it drew on, so grounding and citation can be checked; the pipeline details, so a failure can be traced to the stage where it arose, whether retrieval, synthesis, or formatting; and a golden response, an expert-defined benchmark answer locked in before the system is run.

The dataset is designed rather than sampled at random, structured along three axes so that it spans real usage: the feature being tested, the scenario or problem type, and the persona, the intended reader the output must serve. Designing across these axes, and seeding adversarial cases that target suspected weaknesses, ensures the set probes boundaries rather than confirming easy successes. The working size is roughly 50 to 100 traces, but the real stopping rule is theoretical saturation: continue until new traces stop surfacing new types of failure.

Step 3: Binary evaluation with structured annotations

Each output, or a specific section of it, is judged Pass or Fail against its golden answer. A binary verdict is deliberate: an output either meets a stated expectation or it does not. This avoids the

ambiguity of 1-to-5 scales, where evaluators disagree on what each number means and averages like 3.2 versus 3.7 give an engineering team nothing to act on.

Because a single label loses nuance, each judgment is paired with a structured annotation: the goal the output was meant to achieve, the verdict, the evidence behind the verdict, the reason it passed or failed in domain terms, the insight about underlying behavior this reveals, and a concrete, testable fix for the build team. This pass is done by the human expert, not an LLM, because only the expert carries the context to know that a described feature does not exist, that a cited figure is absent from the sources, or that a required rule was broken. Each verdict is tied to a specific expectation rather than a vague global judgment of quality, and the reasons are written specifically, so that they can later be categorized and counted.

Step 4: Error analysis and axial coding

Once outputs are annotated, the failures are analyzed as a set rather than one at a time. Each annotation yields one or more open codes, the specific reasons recorded in the why and insight fields, and these are grouped into a smaller number of named axial codes: recurring mechanisms such as weak retrieval, missing required elements, or tone mismatch, each quantified by frequency.

Two practices keep the taxonomy honest. The first categories produced are usually too generic to act on, so they are renamed and split into specific, actionable mechanisms and the open codes are re-mapped onto the refined set. A none-of-the-above category is kept throughout: anything landing there signals the taxonomy is incomplete. Frequency is the primary signal for prioritization, but a rare failure can be severe enough to prioritize regardless of count, which is why clinical or safety-critical misses are weighted separately from their raw frequency.

Aligning an LLM-as-Judge

Once the taxonomy stabilizes, an LLM-as-Judge can be aligned to replicate the expert's Pass or Fail judgments for the high-frequency modes, scaling the evaluation beyond what one expert can label by hand. The objective is not a generic quality score but a calibrated evaluator for specific, named failure modes. Where a failure is verifiable in code, such as whether a citation traces to a real source or a required field is present, a deterministic check is preferred over a model. The LLM-as-Judge is reserved for genuinely subjective modes such as tone or reasoning quality, scoped to one mode with a binary output. Raw agreement percentage is not trusted: a lazy judge that always says Pass can score high agreement while catching nothing, so human and model verdicts are compared in a confusion matrix and false positives and false negatives are driven down separately, not just until the headline number looks acceptable.

Step 5: Stakeholder handoff and prioritization

The taxonomy is only useful if it changes what the build team does next, so the final step translates findings into action. The top recurring failure modes are shared with the engineering team, and rather than handing over the full list, two or three high-impact themes are prioritized per iteration. Each is turned into a specific, owned, measurable action: the issue category, its root cause, the concrete change to make, the team responsible, and a way to verify the fix.

Prioritization weighs frequency together with severity, so a high-volume retrieval problem and a rare but safety-critical miss can both earn priority for different reasons. The output of this step is a roadmap, not a report card.

Step 6: Verification and continuous testing

Improvements are confirmed through a layered testing strategy rather than a single check. Scoped unit tests validate mechanical properties, for example that every citation traces back to a verified source or that required structural elements are present. Human-plus-model judging calibrates the LLM-as-Judge against expert verdicts on the subjective modes, maintained through the confusion-matrix check. A/B testing compares new and old pipeline configurations against real user preference. Once the taxonomy and calibrated judges exist, the judges can be run online, sampling real production traces on a regular cadence so that the live failure rate per mode becomes a standing quality signal. At that point a full evaluation loop takes on the order of thirty minutes and can run weekly, turning evaluation from a one-off project into a lightweight, repeating habit.

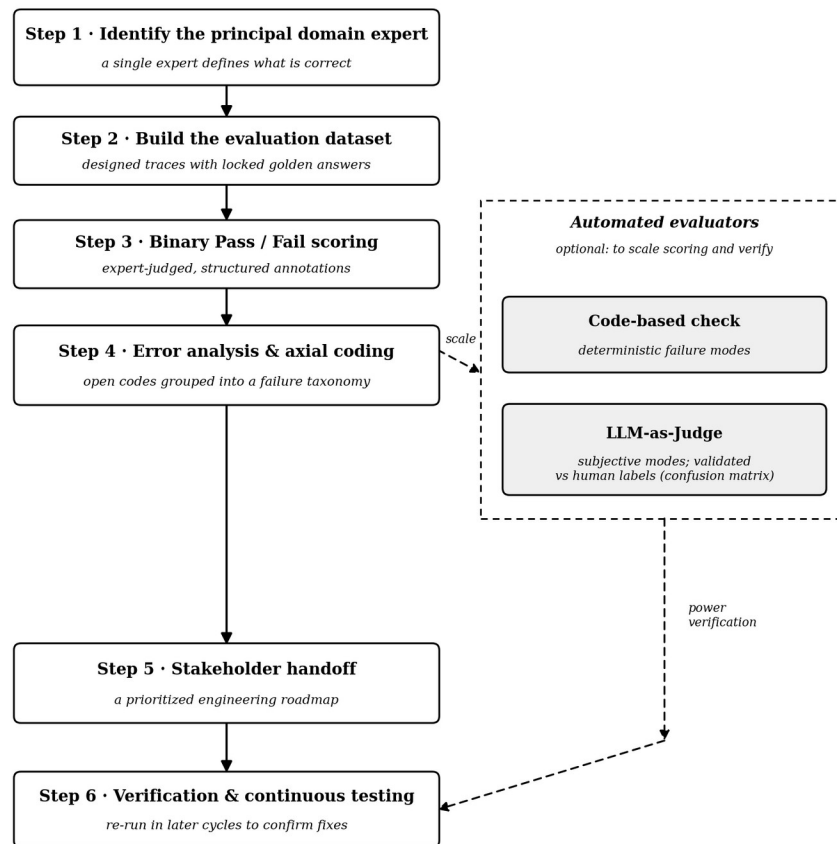


Figure 1. The evaluation method. Six expert-driven steps form the backbone; the automated evaluators (a code-based check and a validated LLM-as-Judge) are optional accelerators that scale scoring and power verification.

4. A Worked Example: KallosumAI

To demonstrate the method on a real, deployed system, it was applied to a product the author built. KallosumAI is a medical-affairs platform that connects physicians and pharmaceutical teams, with two sides. The first is a doctor-facing clinical assistant: physicians ask drug-related questions and receive evidence-backed answers in real time, drawn from a curated literature

corpus and live web search. The second is a pharma-facing analytics layer (a “perception mapper”) that captures the themes and trends emerging from those interactions and turns them into structured intelligence for medical-affairs teams. The platform was shaped by direct user feedback and is the basis for the user evidence discussed later.

This evaluation deliberately covers only one component: the doctor-facing clinical question-answering assistant. The analytics layer is out of scope. This is the right unit to evaluate, because it is the part that makes clinical claims to physicians, where factual grounding, safety, and source credibility carry real consequences. Within that assistant, scope was narrowed further to a single drug, epoetin alfa, drawing on a curated corpus of six peer-reviewed sources combined with live web search. The restriction to one drug is deliberate: a focused, governable knowledge base is what a medical-affairs deployment requires, and is what distinguishes a clinical assistant from a general-purpose search tool.

The six-source corpus spans the principal clinical contexts in which epoetin alfa is used: chronic renal disease (a meta-analysis of clinical end points, and the CHOIR trial on anemia correction), myelodysplastic syndromes (a Phase 3 placebo-controlled trial and a real-world comparative study against darbepoetin alfa), chemotherapy-related anemia in cancer (a quality-of-life meta-analysis), and critical illness (a trial notable for its thrombotic safety signal). This composition is what makes the source-routing behavior testable: questions whose answers lie inside these six sources should be answered from the corpus, current questions outside their scope should trigger web search, and questions answerable by neither should be declined.

Following the method, the evaluation set was a focused first iteration of 35 questions demonstrating the approach, with expansion toward the 50-to-100 range as the natural next pass. Each question was assigned a locked golden answer and an expected source, scored Pass or Fail by the domain expert, a physician, and every failure was annotated and then axially coded into the taxonomy reported below.

Instantiating the framework's three axes

The question set was constructed along the three design axes the method prescribes. The feature axis became five evaluation dimensions: factual grounding, clinical safety, completeness, source credibility and verifiability, and latency. The scenario axis was captured by an answerability classification, marking each question as answerable from the corpus, answerable only by current web search, partially answerable, or answerable by neither, the last category seeded with adversarial traps such as fabricated studies, wrong-premise prompts, and near-miss drug names. A third axis, source routing, recorded where each answer should originate, making the assistant's retrieval decisions directly scoreable. The persona throughout was the practicing physician posing a point-of-care question, which fixed the expected register of a correct answer. Questions were written deliberately against this matrix rather than sampled, so that the set probed boundaries, including patient-safety red flags embedded in otherwise routine queries, rather than confirming easy successes.

Results

Of the 35 questions, 23 passed and 12 failed at the question level. Pass rates varied markedly by dimension (Figure 2): clinical safety was strongest at 80 percent, followed by completeness at 71 percent and source credibility at 60 percent, while factual grounding was weakest at 50 percent.

Latency, the fifth dimension, was captured per answer but analyzed separately and is not included in this pass-rate comparison. Because the dimension subsets are small, these rates are read as directional rather than precise; the value of the breakdown is in where it points, not in the decimal. What it points to is consistent and, for a retrieval-augmented system, diagnostic: the assistant was most reliable on the judgments that carry the highest clinical stakes and least reliable on getting verified facts out of its own knowledge base.

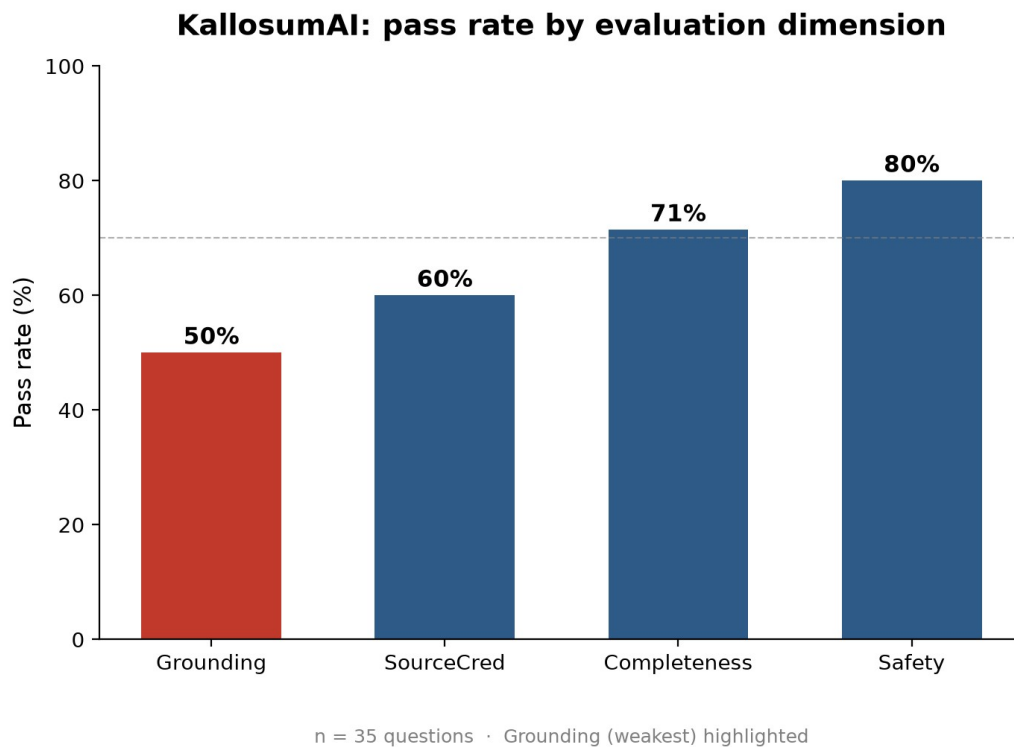


Figure 2. KallosumAI pass rate by evaluation dimension (n = 35 questions). Grounding, the weakest dimension, is highlighted.

A representative failure

One trace illustrates the dominant pattern. The physician asked a point-of-care dosing question: a 74-year-old with IPSS low-risk MDS, hemoglobin 9.2, serum erythropoietin 180, and the standard weekly epoetin alfa starting dose. The golden answer was specific and present in the corpus: a weight-based starting dose of 450 IU/kg/week subcutaneously, escalating to 1050 IU/kg at week 8 if there is no response.

The assistant answered: “*the standard starting dose for epoetin alfa in lower-risk MDS is 40,000 units subcutaneously once weekly,*” with supporting notes drawn from general guideline material and a web search whose cited sources included Facebook and Instagram pages. Scored against the method's annotation schema:

- **Goal:** retrieve the specific, corpus-grounded starting dose for the described patient
- **Verdict:** Fail
- **Evidence:** the answer gave a fixed-dose figure and never surfaced the weight-based dose available in the corpus; its citations were non-authoritative web pages, not the source document

- **Why:** the assistant did not query the RAG corpus before searching the web, and returned a generic guideline answer where a specific, document-grounded one existed
- **Insight:** the failure is upstream of reasoning; the system did not reach its own knowledge base, then compounded the miss by answering generically and citing unverified sources
- **Fix:** enforce RAG-first routing for in-corpus questions; prefer the specific corpus value over generic guidance; restrict web fallback to verified scientific sources

This single trace contributes three open codes spanning the two largest failure categories, source routing error and answer-not-specific-enough, with a secondary sourcing problem. It is representative in the literal sense: the same upstream pattern, the assistant failing to reach or extract from its own corpus, accounts for the largest share of failures in the set, and is what drives the low grounding pass rate seen in Figure 2.

A code-based evaluator for source credibility

Not every failure mode requires an LLM to detect. Following the principle that a deterministic check is preferable wherever a failure can be captured in code, source credibility was evaluated programmatically. The 18 web-search answers cited 188 sources in total; a short Python evaluator parsed each cited URL to its domain and applied two checks. The first flagged banned non-credible domains (social media: Facebook, Instagram, LinkedIn, and similar). The second compared each domain against an explicit allowlist of authoritative sources (regulators such as the FDA, peer-reviewed literature and major publishers, trial registries, and clinical guideline bodies), flagging anything outside it as non-authoritative. Domains were matched on the parsed host rather than by substring, to avoid false positives such as a benefits-management domain (optumrx.com) being mistaken for a social platform (x.com).

17 of the 18 web-search answers cited at least one banned social-media source. All 18 cited at least one non-authoritative source. Across all 188 citations, only 15 percent were authoritative; two answers cited none at all. Because the check is deterministic, it is instant, exact, and reproducible, and it makes the case for reaching for code before an LLM: no model variability, no cost, no drift. It also surfaces a problem the manual quality scoring had under-counted, because the human pass judged answer quality holistically and treated sourcing as a secondary note rather than a separately scored axis.

An LLM-as-Judge for factual grounding

Where source credibility could be checked in code, grounding cannot: deciding whether an answer is faithful to its reference and specific enough is a judgment, not a pattern match. This is the case the method reserves for an LLM-as-Judge. A judge was built for the single dimension of factual grounding, prompted to return a binary PASS or FAIL against the question's golden answer, with explicit instructions that a generic answer omitting the specific values the golden requires is not grounded. It was run on the eight grounding questions (Q1 to Q8), which already carried human Pass/Fail labels from the expert scoring, and its verdicts were compared against those labels.

The judge agreed with the human on 5 of 8 answers (62 percent raw agreement). But raw agreement is exactly the metric the method warns against, and the confusion matrix shows why (Table 1). The judge caught every true failure: all four answers the expert failed, the judge also failed (true-positive rate, or coverage of real failures, 100 percent). Its errors were all in the other

direction. Of the four answers the expert passed, the judge wrongly failed three (true-negative rate 25 percent). In the terms of Shankar et al., the judge has excellent coverage but a high false-failure rate: a first-pass judge that is systematically over-strict, flagging good answers as failures.

	Judge: Fail	Judge: Pass
Expert: Fail	4 (caught)	0 (missed)
Expert: Pass	3 (false alarm)	1 (correct)

Table 1. Grounding judge confusion matrix ($n = 8$): expert labels versus judge verdicts.

The disagreements are diagnostic. The clearest, Q7, was a fabricated-study trap: the assistant correctly refused to confirm a non-existent trial, the expert passed it, and the judge failed it, because the judge prompt defined grounding only in terms of agreement with a factual golden and had no rule for how to treat a correct refusal. This is a textbook instance of what Shankar et al. call criteria drift: the grounding criterion could not be fully specified in advance, because a case the written definition did not anticipate (a correct refusal) only surfaced once real outputs were graded. The fix is not to tune the judge until it agrees, which would fit it to these eight cases, but to extend the criterion to cover refusals and re-validate on a larger labeled set.

Two limitations bound this result. The judge was validated in-sample on eight examples, with no held-out test set, so it demonstrates the alignment workflow rather than a production-ready evaluator. And eight labels are too few to estimate the rates precisely. The value here is the method shown end to end: a narrow binary judge, measured against human labels with a confusion matrix, its failure mode named and explained, and a concrete path to improvement, rather than a single agreement number presented as success.

5. Failure Modes and Engineering Roadmap

Failure modes

Axial coding of the 35 issues surfaced from the evaluation produced ten distinct failure mechanisms, shown ranked by frequency in Figure 3 and detailed in Table 2. The distribution is the central empirical result, and it is concentrated rather than diffuse, which is the paper's core hypothesis borne out: a small number of mechanisms account for most failures, and they cluster in one layer of the system.

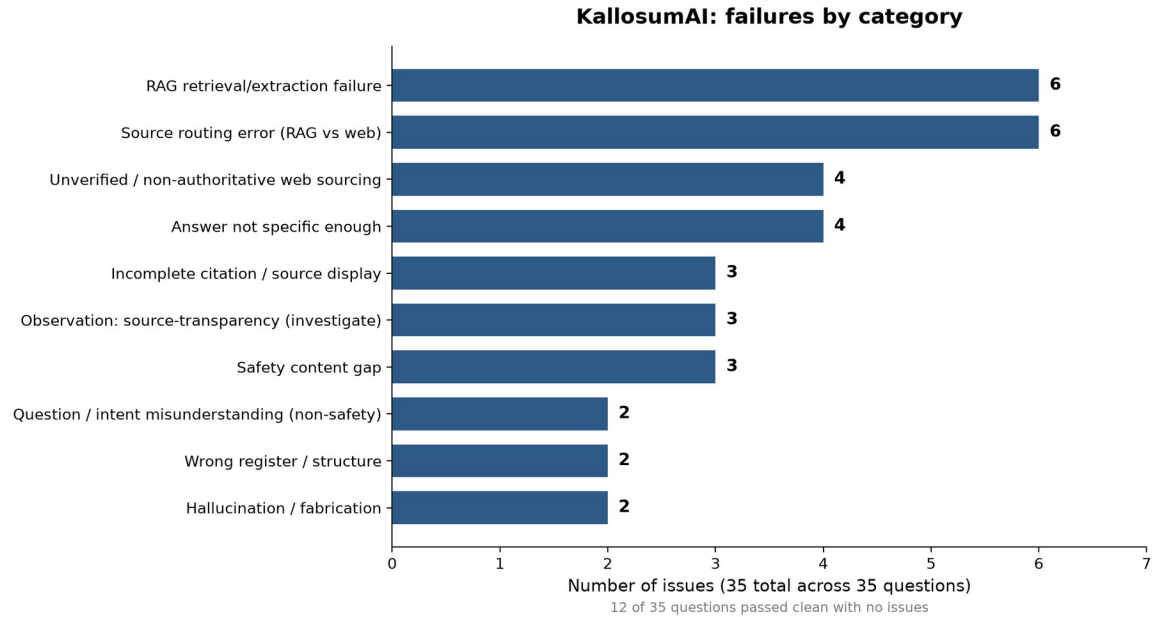


Figure 3. KallosumAI failures by category. 35 issues from 23 of the 35 questions; 12 questions passed clean with no issues.

Issue category	Root cause	Representative example	Count	Insight / fix direction
Source routing error (RAG vs web)	Assistant does not query the corpus before web search	Web-searched a dosing answer that was in the corpus (Q1)	6	Enforce RAG-first routing for in-corpus questions
RAG retrieval / extraction failure	Reached the corpus but could not extract the precise answer	Returned only p-values for a QoL question	6	Improve in-article extraction and chunking
Answer not specific enough	Generic guidance instead of the available specific answer	Gave a fixed dose where a weight-based one existed (Q1)	4	Prefer specific corpus values over generic guidance
Unverified / non-authoritative sourcing	No source whitelist on web fallback	Cited Facebook and Instagram pages	4	Restrict web fallback to verified scientific sources
Safety content gap	Failed to surface a contraindication or warning	Missed curative-intent ESA caution (Q11)	3	Add safety-rule checks for high-risk scenarios
Source-transparency (investigate)	Cannot tell which sources were actually used	Unclear whether social media was relied on	3	Expose sources used so they can be audited
Incomplete citation / source display	Citations lack full study identity	Cited a journal and year with no article name	3	Emit full citation for corpus answers
Hallucination / fabrication	No guard against fabricated specifics	Invented a non-existent dose conversion	2	Suppress fabricated studies and conversions
Wrong register / structure	Output not matched to the physician reader	Academic trial detail instead of practical answer	2	Condition output to the point-of-care persona
Question / intent misunderstanding	Misread non-safety questions	Returned a default fallback to a clear question	2	Improve robustness to phrasing variants

Table 2. Failure taxonomy with representative examples and fix directions.

Read together, the top three mechanisms (source routing, retrieval and extraction, and answer specificity) are all facets of a single underlying problem: the assistant frequently fails to reach, extract from, or faithfully report its own curated corpus. Sixteen of the 35 issues, nearly half, sit in this retrieval layer. A second theme concerns the trustworthiness of web fallback, and here the code-based evaluator quantified the problem precisely: 17 of 18 web-search answers cited a non-credible social-media source, and only 15 percent of all web citations were authoritative. This is the kind of high-prevalence, deterministically measurable failure that the manual taxonomy flagged qualitatively and the automated check then confirmed exhaustively, across every cited source. Safety, hallucination, and citation are lower in frequency but, as the next section argues, not lower in priority.

Frequency is not the only signal

Ranking by count answers what fails most, but for a clinical tool it must be weighed against what fails worst. The safety-content-gap category illustrates the distinction. It appeared only three times, placing it in the middle of the frequency ranking, yet its failures are the most consequential in the entire set. In one trace the physician asked about starting epoetin alfa for fatigue in a patient on curative-intent chemotherapy. Erythropoiesis-stimulating agents carry a boxed warning for shortened survival and tumor progression and are avoided when the treatment goal is cure, so the correct response is to flag that the drug should not be started. The assistant instead returned a generic, helpful-sounding answer and did not surface the contraindication. The failure is rare in the data but, acted upon, could harm a patient. A frequency-only prioritization would rank it below the routing and extraction problems; a clinically informed prioritization does not.

Prioritization and handoff

Table 3 translates the taxonomy into a prioritized set of actions, weighting each mechanism by severity as well as count. Owners are indicative roles rather than a specific team assignment for this worked example.

Priori ty	Issue category	Root cause	Action	Owner	Verification (next cycle)
P0	Safety content gap	No safety-rule check for high-risk scenarios	Add contraindication / boxed-warning checks for safety-critical query patterns	Eng + clinical expert	Re-run the safety question set; confirm fewer unflagged safety misses than the previous cycle
P0	Source routing (RAG vs web)	Corpus not queried before web fallback	Enforce RAG-first routing for in-corpus questions	Eng / retrieval	Re-run the routing-tagged set; confirm more in-corpus questions are answered from the corpus
P0	RAG retrieval / extraction	Cannot extract the precise answer from a reached source	Improve chunking and in-article extraction	Eng / ML	Re-score the in-corpus subset; confirm more questions return the specific corpus answer
P1	Unverified / non-authoritative sourcing	No whitelist on web fallback	Restrict web fallback to verified scientific sources; block social media	Eng / retrieval	Audit cited web sources; confirm fewer non-authoritative citations
P1	Answer not specific enough	Generic guidance preferred over	Prefer specific, document-grounded answers	Prompt / retrieval	Re-score the in-corpus subset; confirm more

Priori ty	Issue category	Root cause	Action	Owner	Verification (next cycle)
		specific corpus value			answers give the specific value over generic guidance
P2	Incomplete citation / transparency	Sources not fully identified or exposed	Emit full citation and expose sources used	Eng	Re-check answers; confirm more carry a complete, verifiable citation
P2	Hallucination / fabrication	No guard against fabricated specifics	Add refusal behavior for non-existent studies and conversions	Prompt / Eng	Re-run the trap set; confirm fewer fabricated responses

Table 3. Prioritized engineering roadmap.

Verification is expressed as a direction of improvement rather than a fixed numeric target, because a single 35-question iteration does not establish a precise enough baseline to promise a specific gain. Each fix is validated by re-running the relevant subset in the next cycle and confirming the failure rate has moved in the intended direction. The result is the artifact the method is designed to produce: not a score, but a ranked, owned, and measurable set of changes, each traceable back to specific failed traces and each checkable in the next evaluation cycle.

6. Discussion and Limitations

The findings converge on a single conclusion: KallosumAI's failures are concentrated in the retrieval layer, not in reasoning. The assistant is most reliable on the highest-stakes judgments (clinical safety) and least reliable at reaching, extracting from, and faithfully citing its own corpus. This is an encouraging result for remediation, because retrieval and routing problems are more tractable than reasoning failures: they point to specific, buildable fixes rather than diffuse model limitations.

Several limitations bound these claims, and naming them is part of the method. First, sample size: the evaluation is a focused first iteration of 35 questions, and per-dimension subsets are small (8 to 10 questions each), so pass rates are directional rather than precise. The methodology specifies expansion toward 50 to 100 traces, guided by theoretical saturation, as the next iteration. Second, single evaluator: scoring was performed by one domain expert. This is a deliberate design choice, since a single principal expert keeps the standard consistent, and the LLM-as-Judge is the intended path to scaling beyond one rater, but inter-rater reliability has not yet been measured. Third, in-sample judge validation: the grounding judge was validated on the same eight labeled examples it was demonstrated on, with no held-out set, so it evidences the alignment workflow rather than a production-ready evaluator. Fourth, single-drug scope: the assistant and evaluation cover epoetin alfa only. This is intentional, since a focused, governable knowledge base is what a medical-affairs deployment calls for, but the failure rates should not be generalized to other drugs without re-evaluation.

Finally, the evaluation does not sit in isolation from the product. KallosumAI was shaped by direct clinician feedback: physicians asked for faster input, visible evidence, and portable output, and the assistant shipped voice input, source references on every answer, and chat export in response. Medical-affairs colleagues independently noted the platform's potential value for field teams, citing its structured, reference-backed answers. The evaluation reported here is the next turn of that same loop: it converts user-facing quality into an engineering roadmap, and the

verification column of that roadmap defines how the next iteration will be measured. The contribution is a method that connects real user feedback, rigorous failure analysis, and a prioritized path to improvement in a single cycle.

7. Conclusion

This worked example supports the paper's central claim: the failures of a clinical retrieval-augmented assistant are not random but concentrate in a small number of identifiable, addressable mechanisms, and a structured, expert-anchored evaluation surfaces them and converts them into an engineering roadmap. For KallosumAI, the dominant finding was that the assistant fails upstream of reasoning, by not reaching, extracting from, or faithfully reporting its own curated corpus, with a secondary theme around the trustworthiness of web fallback and a small but clinically serious set of safety misses that warrant priority despite their low frequency.

The value demonstrated here is the method rather than the scores. Both layers of automation were shown end to end: a deterministic code-based evaluator that quantified the sourcing problem exactly, and an LLM-as-Judge for grounding that, validated against human labels, revealed itself as an over-strict first-pass judge, demonstrating the alignment workflow and the necessity of the confusion matrix over raw agreement. The same procedure applies to any expert-defined domain, and the evaluation that found these problems is the instrument that will verify their fixes: re-running the relevant question subsets in each cycle turns evaluation into a standing quality signal rather than a one-off audit. The immediate next steps follow the method's own later stages: expanding the question set toward the fifty-to-one-hundred range guided by saturation, extending the grounding judge to handle cases such as correct refusals and re-validating it on a larger held-out set, and closing the loop with a build team against the prioritized actions.

References

- Husain, H. (2024). Your AI Product Needs Evals. <https://hamel.dev/blog/posts/evals/>
- Husain, H. (2026). LLM Evals: Everything You Need to Know. <https://hamel.dev/blog/posts/evals-faq/>
- Shankar, S., Zamfirescu-Pereira, J.D., Hartmann, B., Parameswaran, A.G., and Arawjo, I. (2024). Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences. Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24). <https://doi.org/10.1145/3654777.3676450>